

Mass Entity Crowds

Ten thousand agents and more, pathfound on the GPU through the Mass ECS.

IN SHORT

Drop the **LCM Nav3D Agent** trait (`ULcmMassNav3DTrait`) on a Mass entity config, set a goal on each entity's `FLcmMassPathRequestFragment` , and LCM Nav3D pathfinds the whole archetype on the GPU: tens of thousands of agents share batched GPU dispatches instead of paying a UActor tick each. Three auto-registered processors run the request → GPU solve → result → steering loop; you never call the solver yourself.

NOTE

Audience: tech leads evaluating LCM Nav3D for crowd-scale 3D navigation. This tutorial covers the full shipped surface: the foundation (fragments/processors/bridge), the **flow-field crowd path** for 10K shared-goal swarms, **natural movement** (pure-pursuit + turn-rate/accel limits + flight style + ORCA + lane spreading), and the **per-agent CPU/GPU + solver + smoothing** options that mirror the BehaviorTree FlyTo surface. Where a piece is still editor-bound (Niagara `.uasset` , pre-built sample `.umap` s), it says so explicitly.

NOTE

Two crowd modes, pick one per agent. (1) **Per-agent path:** the *LCM Nav3D Agent* trait; each agent computes its own path (the BT-equivalent at ECS scale; flexible, supports arbitrary distinct goals). (2) **Shared flow field:** the *LCM Nav3D Flow Steer* trait; one CPU SVO flow field is built per goal and sampled O(1) by the whole crowd (the natural 10K-scale path for a shared goal). They are mutually exclusive, so use one trait or the other.

Why Mass

A `UActor` AI agent costs ~50 µs/tick once you add a controller, movement component, and behaviour tree. A few hundred of them saturate the game-thread frame budget long before the GPU pathfinder breaks a sweat. UE's Mass framework represents agents as ECS-style fragments in cache-dense archetypes, so a single processor sweep iterates tens of thousands of entities at sub-microsecond per-entity cost. With Mass in front of it, **LCM Nav3D's GPU dispatch becomes the binding constraint instead of the UActor layer**, which is what makes the 10K+ agent number reachable.

Prerequisites

- UE 5.8, LCMNav3D plugin enabled.
- The `MassEntity` and `MassGameplay` plugins enabled. LCMNav3D enables both (see `LCMNav3D.uplugin`); if you stripped them, re-enable both or the module fails to load.
- An `ALcmNavigationManagerSVO` in the level with a built SVO (the same manager every LCM Nav3D path query uses).
- Familiarity with the standard Mass authoring flow (config asset + spawner). If you've never used Mass, do Epic's "MassEntity Quick Start" first: LCM Nav3D only adds a trait and three processors on top of stock Mass.

The shipped surface

Type	Role
<code>ULcmMassNav3DTrait</code> (trait: "LCM Nav3D Agent")	The single authoring entry point for the per-agent path mode. Attaches the nav fragments + exposes the full option surface: <code>AgentRadius</code> , <code>MaxSpeed</code> , <code>MobilityClassIndex</code> , avoidance radius/strength, pathfinding (<code>bUseGPUPathfinding</code> , <code>PathSolver</code> , <code>SmoothingMode</code>) and movement (<code>MaxAcceleration</code> , <code>MaxTurnRateDeg</code> , <code>LookAheadDistance</code> , <code>ArriveRadius</code> , <code>FlightStyle</code> , <code>AvoidanceStyle</code> , <code>AvoidanceTimeHorizon</code> , <code>LaneBias</code> , <code>LaneDensityNorm</code>).
<code>ULcmMassFlowSteerTrait</code> (trait: "LCM Nav3D Flow Steer")	The authoring entry point for the shared-flow-field crowd mode (10K-scale). Carries the goal + speed + avoidance; self-contained (shares no fragment with the path trait).
<code>FLcmMassNavAgentFragment</code>	Per-agent params + runtime <code>Velocity</code> + the full pathfinding/movement option surface (set by the trait, read by the bridge + steerer).
<code>FLcmMassPathRequestFragment</code>	The request slot: <code>Goal</code> , <code>Status</code> (<code>ELcmMassPathStatus</code>), plus internal <code>RequestId</code> / <code>LastDispatchFrame</code> .
<code>FLcmMassPathResultFragment</code>	Steering-facing result: <code>NextWaypoint</code> , <code>CurrentWaypointIndex</code> , <code>WaypointCount</code> .
<code>ULcmMassPathRequestProcessor</code> (PrePhysics)	Collects <code>Pending</code> requests (budgeted by <code>r.LcmNav.Mass.MaxPathsPerTick</code>), hands them to the bridge.
<code>ULcmMassPathFollowingProcessor</code> (DuringPhysics)	The natural path follower : pure-pursuit look-ahead → density-aware lane spreading → avoidance (boids/ORCA per <code>AvoidanceStyle</code>) → flight style (LinearDirect / PhysicsWeighted turn-rate+accel) → arrival.
<code>ULcmMassPathResultProcessor</code> (PostPhysics)	Drains completed paths, writes the result fragment, advances waypoints.
<code>ULcmMassFlowSteerProcessor</code> (DuringPhysics)	The flow-field steerer for the shared-flow-field mode: samples the cached field O(1) per agent + lane spreading.

Type	Role
<code>ULcmMassFlowFieldSubsystem</code> (world subsystem)	Generates + caches one CPU SVO flow field per goal (the shared-flow-field mode).
<code>ULcmMassAgentBridgeSubsystem</code> (world subsystem)	The Mass↔LCM Nav3D dispatch bridge. Routes each request through <code>FindPathAsync</code> (CPU async) or GPU batched per the agent's <code>bUseGPUPathfinding</code> . Internal.

All three processors **auto-register** with the Mass processing phases: once an entity has the trait's fragments, it is picked up automatically. There is no per-agent component, tick function, or controller.

Step 1: Create the agent config

- Content Browser → Add → Miscellaneous → Data Asset → `MassEntityConfigAsset`. Name it `DA_MassAgent_Apex`.
- Open it → Traits → + → add LCM Nav3D Agent.
- Set the trait parameters. **Core:** `AgentRadius` (cm, default `50`), `MaxSpeed` (cm/s, default `600`), `MobilityClassIndex` (`0`). `LCM Nav3D|Mass|Pathfinding` (mirrors the BT FlyTo surface):
 - `bUseGPUPathfinding`: CPU async (default, stable, scales) vs GPU batched dispatch.
 - `PathSolver`: `LazyThetaStar` (default, any-angle → smooth diagonals) vs `AStar` (grid-locked → square turns).
 - `SmoothingMode`: `CatmullRom` (default, curved) / `Linear` (shortcut) / `None` (raw staircase). `LCM Nav3D|Mass|Movement` (the natural-movement layer):
 - `MaxAcceleration` (cm/s²), `MaxTurnRateDeg` (deg/s, the knob that turns square corners into arcs), `LookAheadDistance` (cm, pure-pursuit), `ArriveRadius` (cm).
 - `FlightStyle`: `PhysicsWeighted` (default, inertia: turn-rate + accel) vs `LinearDirect` (magic/biological, instant).
 - `AvoidanceStyle`: `Reactive` (boids, default) / `Predictive` (ORCA, smooth mutual passing) / `None` / CBS. `AvoidanceTimeHorizon` tunes ORCA.
 - `LaneBias` (default `0.6`) + `LaneDensityNorm` (`6`): density-aware lateral lane spreading so crowds fan onto distinct lanes instead of balling up at turns.

NOTE

Defaults (LazyThetaStar + CatmullRom + PhysicsWeighted + Reactive) give natural movement out of the box. Set `MaxTurnRateDeg` / `MaxAcceleration` to `0` for the legacy instant-snap behaviour.

4. (Optional) Add Epic's stock **Mass Movement** / representation traits if you want the standard movement-or-LOD behaviour alongside LCM Nav3D navigation. The LCM Nav3D trait only adds navigation state; it composes with the rest of the Mass trait library.

NOTE

The trait also adds a `FTransformFragment` if your template doesn't already have one. LCM Nav3D steers that transform.

Step 2: Spawn the agents

Use the standard Mass spawner flow:

1. Place a `MassSpawner` actor in the level.
2. In its **Entity Types**, reference `DA_MassAgent_Apex`.
3. Set a **Spawn Count** (start with 100 while you wire things up; push to 10 000 once it works).
4. Give it a spawn-distribution data asset (Epic's `MassSpawnLocationProcessor` / a `EnvQueryGenerator` location, or a simple grid) so agents start at distinct positions inside your SVO volume.

Hit Play. You now have N LCM Nav3D-Mass entities sitting idle (they have no goal yet).

Step 3: Give the agents a goal

Turnkey (no code): add the "LCM Nav3D Mass Goal" trait (`ULcmMassGoalTrait`) alongside the LCM Nav3D Agent trait. Set **Goal Mode** to either **Fixed Location** (set `GoalLocation`) or **Tagged Actor** (set `TargetActorTag`, e.g. `ApexGoal`, and tag a target actor in the level). Every agent paths to it automatically and re-paths if a moving tagged target drifts past `RepathToleranceCm`; it stops re-pathing once within `ArrivalToleranceCm`. (Goal must be in the agent's SVO chunk: intra-chunk foundation.) That's the fastest way to see agents move.

Under the hood, and for full control, an agent navigates when something writes a `Goal` and flips `Status` to `Pending` on its `FLcmMassPathRequestFragment`. From there the processors own every transition. Three common ways to do it:

A. From a StateTree (recommended for per-entity AI). LCM Nav3D's StateTree integration ([Tutorial 04](#)) runs per-entity via Epic's `UMassStateTreeProcessor`. A StateTree task sets:

```
RequestFragment.Goal = ChosenGoalWorldPos;
RequestFragment.Status = ELcmMassPathStatus::Pending;
```

B. From a tiny custom processor (e.g. "everyone heads to the objective"):

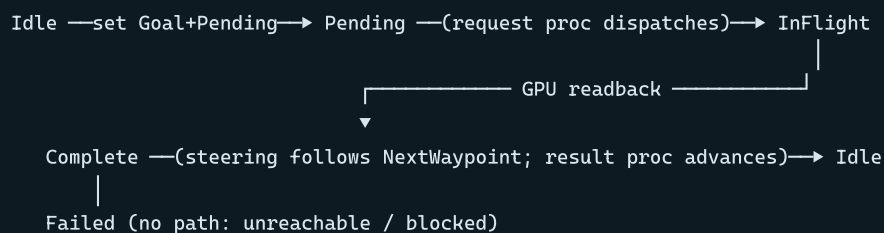
```
void UMyGoalAssignProcessor::Execute(FMassEntityManager& EM, FMassExecutionContext& Ctx)
{
    EntityQuery.ForEachEntityChunk(EM, Ctx, [this](FMassExecutionContext& C)
    {
        auto Reqs = C.GetMutableFragmentView<FLcmMassPathRequestFragment>();
        for (int32 i = 0; i < C.GetNumEntities(); ++i)
        {
            if (Reqs[i].Status == ELcmMassPathStatus::Idle)
            {
                Reqs[i].Goal = ObjectiveLocation;
                Reqs[i].Status = ELcmMassPathStatus::Pending;
            }
        }
    });
}
```

C. From gameplay code holding an `FMassEntityHandle` :

```
auto& Req = EntityManager.GetFragmentDataChecked<FLcmMassPathRequestFragment>(Handle);
Req.Goal = TargetPos;
Req.Status = ELcmMassPathStatus::Pending;
```

That's all. The request processor dispatches it on the next pre-physics phase; the result processor writes `NextWaypoint` once the GPU returns; the steering processor moves the transform along the path.

The lifecycle (what the processors do for you)



`ELcmMassPathStatus` is readable on the request fragment at any time, so your AI logic can branch on `Complete` / `Failed`.

Step 4: Read the result (optional)

If you want your own steering instead of the built-in `ULcmMassPathFollowSteeringProcessor`, read the result fragment:

```
const auto& Res = EntityManager.GetFragmentDataChecked<FLcmMassPathResultFragment>(Handle);
if (Res.WaypointCount > 0)
{
    const FVector Steer = (Res.NextWaypoint - CurrentPos).GetSafeNormal();
    // apply Steer to your own movement
}
```

Tuning

CVar	Default	What it does
<code>r.LcmNav.Mass.MaxPath sPerTick</code>	64	Caps how many <code>Pending</code> requests the request processor dispatches per tick. Paths are async (non-blocking); this just spreads a large dispatch wave over frames.
<code>r.LcmNav.Mass.UseGPU</code>	-1	Global override for the per-agent <code>bUseGPUPathfinding</code> . -1 = use each agent's trait flag; 0 = force CPU async for all; 1 = force GPU batched for all.
<code>r.LcmNav.Mass.Avoidan ce</code>	1	Master switch for inter-agent avoidance in the path follower (per-agent style/radius come from the trait).
<code>r.LcmNav.Mass.LaneBia s / .LaneDensityNorm</code>	-1	Global overrides for the per-agent lane-spread params (<0 = use per-agent).
<code>r.LcmNav.Mass.Flow*</code>	-	Flow-field-mode knobs: <code>FlowAvoidance</code> , <code>FlowGoalQuantCm</code> , <code>FlowLaneBias</code> , <code>FlowLaneDensityNorm</code> , <code>FlowArriveCm</code> .
<code>r.LcmNav.Mass.DebugDr aw</code>	0	Status-coloured debug spheres + waypoint lines (both modes).

These are advanced knobs / kill-switches: production authoring is the trait, not the console.

"I added the trait but nothing spawned!"

This is the most common first-run trap. The LCM Nav3D Agent trait is navigation-only; it adds no visual representation. Entities spawned with only this trait *do* exist, they're just invisible (UE's default/debug Mass configs include a representation trait, which is why those "show up"). Three ways to deal with it:

1. **Confirm they spawned:** in PIE open the **Gameplay Debugger** (`'`) → Mass category, or check `LogMass`; the entity count is non-zero.
2. **See them immediately (no asset needed):** set `r.LcmNav.Mass.DebugDraw 1` in the console. LCM Nav3D draws a debug sphere per agent (coloured by path status: grey Idle / yellow Pending / orange InFlight /

green Complete / red Failed) and a line to its current waypoint. This is a debug aid, not a shipping renderer. Leave it `0` in builds.

3. **Real visuals, turnkey (recommended to start):** add the "LCM Nav3D Mass ISM Render" trait (`ULcmMassNavISMRenderTrait`) to the *same* config and set its **Mesh** (+ optional **Uniform Scale**). Every agent is drawn as an instanced static mesh at its transform: **no LOD/viewer config**, which is what makes Epic's `Visualization` trait so often show nothing. Good for hundreds to low-thousands of agents.
4. **Real visuals, stock / scale:** Epic's `Visualization` trait (`UMassVisualizationTrait`) gives LOD-managed ISM rendering but needs mesh + LOD-distance + viewer config; the GPU-instanced Niagara renderer is the 10K+ path. The LCM Nav3D nav trait composes with all of these; it deliberately owns navigation only.

NOTE

Counts note: the LCM Nav3D ISM Render trait rebuilds its instance buffer each frame (turnkey, fine for modest crowds). For 10K+ use Niagara.

Alternative: shared flow-field crowd mode (10K-scale)

When a whole crowd heads to the **same goal**, computing one path per agent is wasteful (N queries). Use the "LCM Nav3D Flow Steer" trait (`ULcmMassFlowSteerTrait`) *instead of* the LCM Nav3D Agent trait:

1. Add **LCM Nav3D Flow Steer** to the config (do **not** also add LCM Nav3D Agent; they are mutually exclusive).
2. Set `Goal` (world space), `MaxSpeed`, `AgentRadius`, `AvoidanceRadius` / `AvoidanceStrength`.
3. Spawn + Play. `ULcmMassFlowFieldSubsystem` builds **one** CPU SVO flow field toward the goal (eikonal cost field from the goal leaf + per-leaf gradient direction), caches it, and every agent samples its leaf's direction in **O(1)** and steers, so the pathfinding cost is one field build amortised across the whole crowd, not N path queries. CPU + synchronous: no GPU dispatch, no per-agent async wiring.

Density-aware **lane spreading** (`r.LcmNav.Mass.FlowLaneBias`) makes the crowd fan across a corridor onto distinct lanes instead of balling up at turns, and an arrival stop (`FlowArriveCm`) prevents a shove-pile at the goal. Scope (v1): intra-chunk, ideal for **finite worlds** (one SVO = the whole world → one field covers a 10K crowd). Verified by `LcmMassFlowField` (descent property: every leaf steers downhill to the goal).

Scale + rendering

- **GPU pathfinding scales to the agent count:** `ULcmMassScaleBenchmarkCommandLet` measures per-agent amortised wall (dispatch + GPU-solve + readback, ÷ N) via direct `FindPathsGPU_Batched` dispatch, wave-paced to stay under the GPU watchdog. Measured (dev GPU, 3 repeats, intra-chunk): **N=1000 → 0.116 ms/agent; N=10000 → 0.1215 ms/agent, all 10,000 returning, i.e. flat per-agent cost from 1K to 10K (clean linear scaling, no degradation at scale).** The G8 *hard* thresholds (≤ 0.20 ms @1K, ≤ 0.10 ms @10K) are calibrated for the RTX-5090 reference: the 1K gate passes here, the 10K gate (0.1215 vs 0.10) is a

hardware-relative miss on a non-reference GPU, not a scaling defect. Run on the reference hardware for the certified PASS.

- **Rendering 10K+ agents** needs GPU-instanced visuals: `UActor` meshes collapse long before the pathfinder does. Use the **LCM Nav3D ISM Render** trait for instanced-mesh rendering, or Epic's stock **Visualization** trait for LOD-managed rendering; for very large crowds, drive a Niagara system from the per-entity transforms the steering processor writes.

Composition with the rest of LCM Nav3D

- **StateTree**: per-entity AI logic lives in a StateTree driven by `UMassStateTreeProcessor`; the StateTree writes `Goal` + `Pending` and branches on `Status`. Navigation stays entirely in LCM Nav3D.
- **Flow-field crowd mode (shipped)**: for a shared-goal crowd, the *LCM Nav3D Flow Steer* trait is the 10K-scale path: one CPU SVO flow field per goal, sampled O(1) per agent (see the dedicated section above). Cross-chunk GPU flow fields for infinite multi-chunk crowds are the follow-up.
- **Dynamic obstacles (RCU)**: dynamic-obstacle writes are RCU-safe against in-flight Mass reads; see the verification below.

Honest scope (foundation v1)

- The bridge currently resolves both endpoints **within the agent's start chunk** (intra-chunk A*). Multi-chunk macro-route stitching for Mass agents is a follow-up; for now, keep Mass-agent goals within the same chunk as the agent, or use the UActor path for long cross-chunk routes.
- This tutorial builds its sample agent and crowd from scratch, so everything here works with the content that ships in the package.

Verify it yourself (headless, ships today)

Four commandlets exercise the foundation end-to-end. Run them to confirm the integration before committing to it:

```

:: Does a Mass entity actually pathfind end-to-end?
UnrealEditor-Cmd.exe <Project>.uproject -run=LcmMassSmokeTest -AllowCommandletRendering -unattended -
nopause -nosplash

:: Does a Mass path match the UActor path bit-for-bit?
UnrealEditor-Cmd.exe <Project>.uproject -run=LcmMassFixtureRegression -AllowCommandletRendering -
unattended -nopause -nosplash

:: Are reads RCU-safe under concurrent dynamic-obstacle writes?
UnrealEditor-Cmd.exe <Project>.uproject -run=LcmMassRCUHammer -AllowCommandletRendering -unattended -
nopause -nosplash

:: Stable under sustained spawn/despawn churn (no leak/slowdown)?
UnrealEditor-Cmd.exe <Project>.uproject -run=LcmMassSoak -AllowCommandletRendering -unattended -nopause -
nosplash

```

Each returns exit code `0` on pass and logs a `RESULTS / PASS` line. Add `-Agents=10000` / `-Seconds=1800` to push them to gate scale on your reference hardware.

Related LCM Nav3D surfaces

- [Tutorial 04: The Same Agent in StateTree](#) (the per-entity AI layer that pairs with Mass).
- `ULcmMassApexBridgeSubsystem` : the dispatch bridge (internal; see `Classes/Mass/`).
- `ULcmMassScaleBenchmarkCommandlet` : the G8 10K-agent scale-curve gate.

Chapter 6 of the LCM Nav3D User Manual.

The complete manual is `LCM_Nav3D_Manual.pdf`, in this folder alongside every other document in the set. Start from `README.pdf`.

LCM Nav3D: True Volumetric 3D Navigation for Unreal Engine. Version 2.05 · Unreal Engine 5.8.

Copyright © 2026 L.Charitha Madhushanka. All Rights Reserved. Proprietary and confidential. Licensed, not sold; use is governed by the licence that accompanies this software. Unreal Engine is a trademark of Epic Games, Inc. All trademarks are the property of their respective owners.